Decision trees and random forests

🎯 Today's goal

By the end of this hour you will:

- understand a **decision tree** as a game of "20 questions" about the data
- train one with `DecisionTreeClassifier` and control its size with `max_depth`
- **read the rules** a tree learned, as text and as a picture
- spot **overfitting** by comparing train and test accuracy
- build a **random forest** (many trees voting) and see which features matter most

🤔 Why this matters

Think of the game "20 questions". Your friend thinks of an animal, and you ask yes/no questions: *"Does it fly?"* No. *"Does it live in water?"* Yes. *"Is it bigger than a person?"* Yes... *"A whale!"* Good players ask the questions that split the possibilities best.

A **decision tree** plays exactly this game with data. It learns a list of questions like *"Is the colour intensity at most 3.8?"* and follows the answers down to a prediction. Trees are popular in banks, hospitals and shops because a person can **read** the rules and check them. A **random forest** is a crowd of trees that vote, and it is one of the most used models for table-shaped data.

🍷 Today's data: the wine dataset

scikit-learn comes with a small classic teaching dataset: chemical measurements of **178 wines** grown in one region of Italy by **three different growers**. The growers are called `class_0`, `class_1` and `class_2`. Our job: from the 13 measurements (alcohol, colour, and so on), predict which grower made each wine. You don't need to know anything about wine; the columns are just numbers.

```
import pandas as pd            # as_frame=True below gives pandas tables
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
X = wine.data          # the features: 13 measurement columns
y = wine.target        # the label: grower 0, 1 or 2

print("Rows and columns:", X.shape)
print("Growers:", ", ".join(wine.target_names))
print("Wines per grower:", y.value_counts().sort_index().to_dict())
# show 4 of the 13 columns for the first 5 wines
X[["alcohol", "flavanoids", "color_intensity", "proline"]].head()
```

```
Rows and columns: (178, 13)
Growers: class_0, class_1, class_2
Wines per grower: {0: 59, 1: 71, 2: 48}
```

	alcohol	flavanoids	color_intensity	proline
0	14.23	3.06	5.64	1065.0
1	13.20	2.76	4.38	1050.0
2	13.16	3.24	5.68	1185.0
3	14.37	3.49	7.80	1480.0
4	13.24	2.69	4.32	735.0

🌳 A tree is just a set of if / else questions

Before using scikit-learn, here is a "tree" written by hand as a function with `if` / `elif` / `else` (Days 4 and 6). It asks at most two questions about a wine and then names a grower:

```
def guess_grower(color_intensity, flavanoids):
    if color_intensity <= 3.8:          # question 1
        return "class_1"
    elif flavanoids <= 1.6:             # question 2
        return "class_2"
    else:
        return "class_0"

first = X.iloc[0]                       # the first wine in the table
print("My guess:", guess_grower(first["color_intensity"], first["flavanoids"]))
print("Real grower:", wine.target_names[y.iloc[0]])
```

```
My guess: class_0
Real grower: class_0
```

Where did the numbers 3.8 and 1.6 come from? I cheated: a decision tree found them. Choosing good questions by hand is hard. The point of today is to let the computer **learn the questions** from examples.

Some words you'll see:

Word	Meaning
node	one question in the tree, like `color_intensity <= 3.8`
root	the very first question, at the top (trees are drawn upside down)
leaf	an end point with no more questions: it gives the prediction
depth	how many questions in a row the tree may ask before reaching a leaf

🤖 Training a decision tree

Same pattern as every scikit-learn model (Day 16): split the data, create the model, `fit`, then `score` on the test set. `max_depth=2` means "ask at most 2 questions in a row", which keeps the tree tiny and easy to read.

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

small_tree = DecisionTreeClassifier(max_depth=2, random_state=42)
small_tree.fit(X_train, y_train)

print(f"Train accuracy: {small_tree.score(X_train, y_train):.3f}")
print(f"Test accuracy:  {small_tree.score(X_test, y_test):.3f}")
```

```
Train accuracy: 0.940
Test accuracy:  0.867
```

Only two questions in a row, and it already gets most of the test wines right. How does the tree choose a question? It tries *every* column and many cut-off values, and keeps the question that leaves the two groups **least mixed** (ideally each group holds just one grower). scikit-learn measures "mixed" with a number called **gini**: 0 means "all one class", bigger means more mixed.

📜 Reading the rules

The best thing about a tree: you can print what it learned. `export_text` turns the tree into indented text. Give it the column names, or it calls them `feature_0`, `feature_1`…

```
from sklearn.tree import export_text

rules = export_text(small_tree, feature_names=list(X.columns))
print(rules)
```

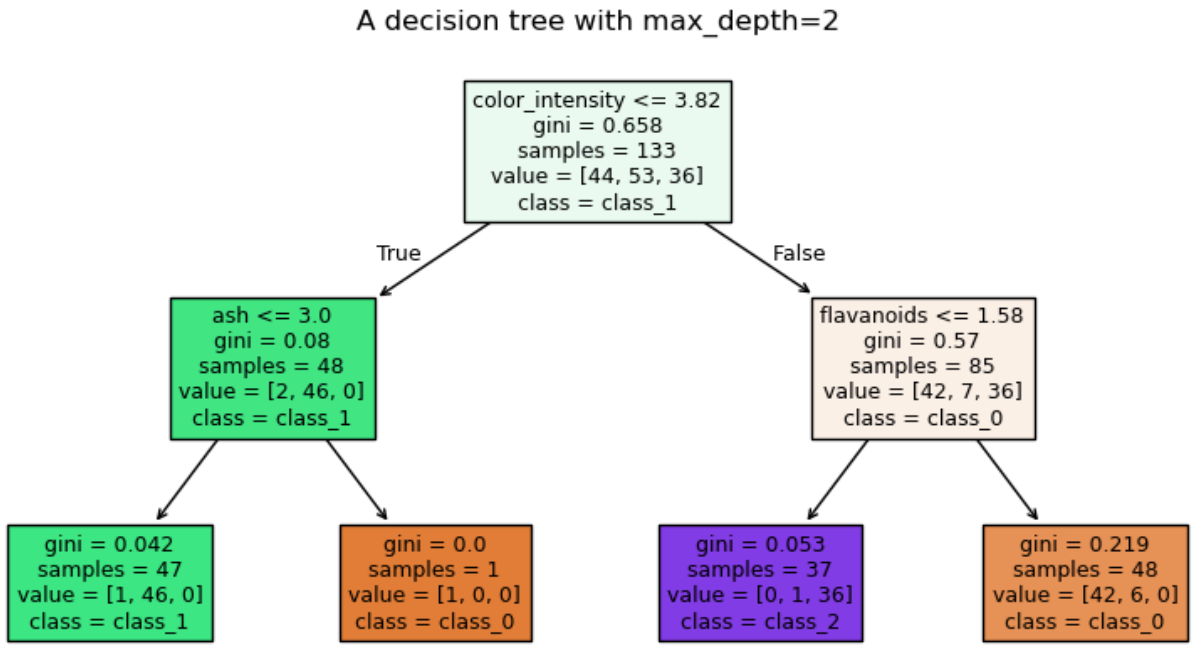
```
|--- color_intensity <= 3.82
|   |--- ash <= 3.00
|   |   |--- class: 1
|   |--- ash >  3.00
|   |   |--- class: 0
|--- color_intensity >  3.82
|   |--- flavanoids <= 1.58
|   |   |--- class: 2
|   |--- flavanoids >  1.58
|   |   |--- class: 0
```

Read it like the `if` / `else` function above: each `|---` is a question, and more indentation means a deeper question. Lines ending in `class: …` are leaves.

You can also draw the tree with Matplotlib (Day 13). Each box shows its question, its gini, how many training wines reached it (`samples`), and the winning class. Colours show the class, and paler boxes are more mixed:

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(10, 5))
plot_tree(small_tree, feature_names=list(X.columns),
          class_names=list(wine.target_names), filled=True, fontsize=9)
plt.title("A decision tree with max_depth=2")
plt.show()
```



To predict for a new wine, the tree simply follows the arrows. Here are the first three test wines:

```
three_wines = X_test.iloc[:3]
print("Predicted:", small_tree.predict(three_wines))
print("Real:     ", y_test.iloc[:3].to_numpy())
```

```
Predicted: [0 1 0]
Real:      [0 1 0]
```

A model must be trained before it can answer. Asking a brand-new tree for predictions is a mistake you will make one day:

💡 This example has a mistake on purpose. Run it and read the error.

```
new_tree = DecisionTreeClassifier(max_depth=2)
new_tree.predict(X_test)         # oops: we forgot new_tree.fit(...)
```

⚠️ **NotFittedError**

```
Traceback (most recent call last):
  line 2, in <module>
    new_tree.predict(X_test)         # oops: we forgot new_tree.fit(...)
    ~~~~~~~~~~~~~~~~^^^^^^^^
sklearn.exceptions.NotFittedError: This DecisionTreeClassifier instance is not
fitted yet. Call 'fit' with appropriate arguments before using this estimator.
```

`NotFittedError` means exactly that: call `.fit(X_train, y_train)` first, then `.predict(...)`.

⭐ Which columns did the tree use?

After training, `feature_importances_` gives every column a score. The scores add up to 1, and a column the tree never asked about gets 0. Putting them in a pandas Series (Day 10) makes them easy to sort and plot:

```
tree_importance = pd.Series(small_tree.feature_importances_, index=X.columns)
tree_importance = tree_importance.sort_values(ascending=False)
tree_importance.round(3)
```

```
flavanoids                    0.492
color_intensity               0.483
ash                           0.026
alcohol                       0.000
malic_acid                    0.000
alcalinity_of_ash             0.000
magnesium                     0.000
total_phenols                 0.000
nonflavanoid_phenols          0.000
proanthocyanins               0.000
hue                           0.000
od280/od315_of_diluted_wines  0.000
proline                       0.000
dtype: float64
```

Only the columns in the questions above get a score above 0. With just two levels of questions, the tree ignores most of the data.

📈 Deeper trees and overfitting

What if we let the tree ask more questions? Let's try several depths in a loop (Day 5) and write down train and test accuracy each time. `None` means "no limit: keep asking until every leaf is pure" (holds wines from only one grower).

```
depths = [1, 2, 3, 4, 5, None]
for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_acc = model.score(X_train, y_train)
    test_acc = model.score(X_test, y_test)
    gap = train_acc - test_acc
    print(f"depth={depth}  train={train_acc:.3f}  test={test_acc:.3f}  gap={gap:.3f}")
```

```
depth=1  train=0.662  test=0.600  gap=0.062
depth=2  train=0.940  test=0.867  gap=0.073
depth=3  train=0.992  test=0.956  gap=0.037
depth=4  train=1.000  test=0.956  gap=0.044
depth=5  train=1.000  test=0.956  gap=0.044
depth=None  train=1.000  test=0.956  gap=0.044
```

Look at the train column: from depth 4 on, the tree scores a **perfect 1.000** on the training wines. It has asked enough questions to give every single training wine its own little corner. But the test score doesn't follow; it stays below 1. That is **overfitting**: the model *memorised* the training examples, including their random quirks, instead of learning the general pattern. It's like a student who learns last year's exam answers by heart and then meets new questions.

Rule of thumb: a **big gap** between train and test accuracy is the warning sign of overfitting. A perfect train score is not a reason to celebrate. Limiting `max_depth` is the simplest cure. On this small, clean dataset the gap stays modest; on messy real data an unlimited tree can do far worse on the test set.

A single tree has another weakness: it is **jumpy**. Train it on a slightly different split and you can get quite different questions and scores. Let's split the data 5 different ways:

```
tree_scores = []
for seed in [0, 1, 2, 3, 4]:
    Xa, Xb, ya, yb = train_test_split(
        X, y, test_size=0.25, random_state=seed, stratify=y)
    model = DecisionTreeClassifier(random_state=42).fit(Xa, ya)
    tree_scores.append(round(model.score(Xb, yb), 3))

print("One tree, 5 different splits:", tree_scores)
```

```
One tree, 5 different splits: [0.889, 0.889, 0.956, 0.933, 0.956]
```

🌲🌲🌲 Random forests: many trees vote

Ask one person to guess the number of sweets in a jar and they may be far off. Ask 100 people and take the average, and the guess is usually much better: the individual mistakes cancel out. A **random forest** uses this "wisdom of the crowd":

1. It trains many trees (`n_estimators` of them, for example 100).
2. Each tree sees a slightly different **random sample** of the training rows, and at every question it may only pick from a random handful of the columns. So the trees end up different from each other.
3. To predict, every tree votes, and the class with the most votes wins.

```
from sklearn.ensemble import RandomForestClassifier

forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)

print(f"Forest train accuracy: {forest.score(X_train, y_train):.3f}")
print(f"Forest test accuracy:  {forest.score(X_test, y_test):.3f}")
print("Number of trees inside:", len(forest.estimators_))
```

```
Forest train accuracy: 1.000
Forest test accuracy:  1.000
Number of trees inside: 100
```

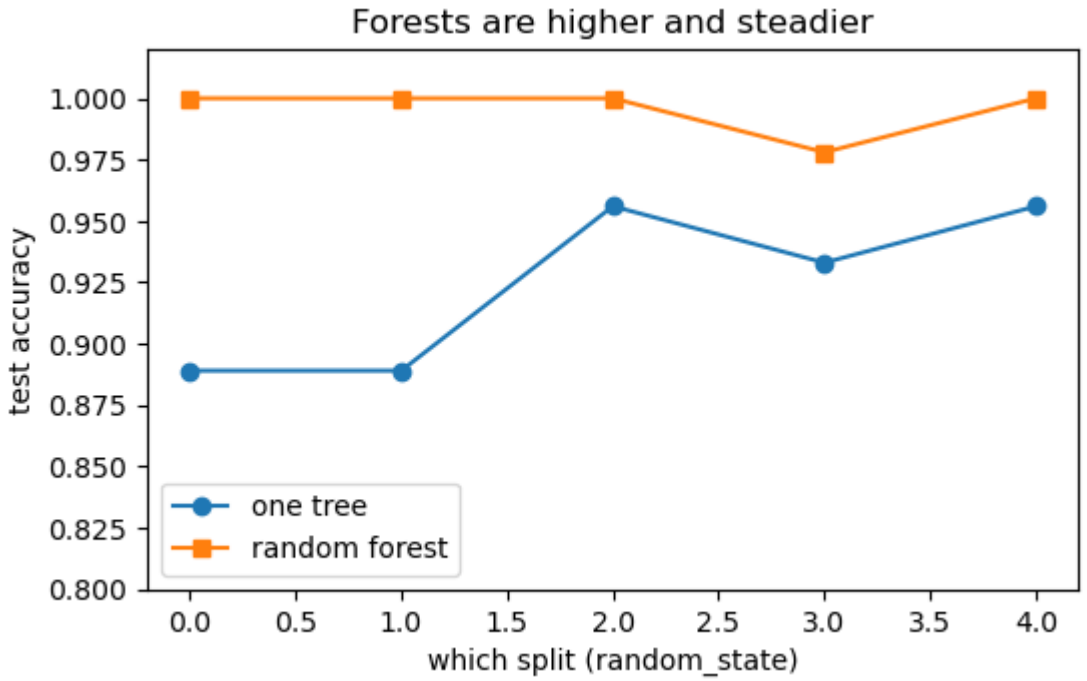
`random_state=42` fixes the random choices, so you get the same forest every time you run the box. Now the same 5-splits experiment, single tree versus forest:

```
forest_scores = []
for seed in [0, 1, 2, 3, 4]:
    Xa, Xb, ya, yb = train_test_split(
        X, y, test_size=0.25, random_state=seed, stratify=y)
    model = RandomForestClassifier(n_estimators=50, random_state=42).fit(Xa, ya)
    forest_scores.append(round(model.score(Xb, yb), 3))

print("One tree:", tree_scores)
print("Forest:  ", forest_scores)

plt.figure(figsize=(6, 3.5))
plt.plot([0, 1, 2, 3, 4], tree_scores, marker="o", label="one tree")
plt.plot([0, 1, 2, 3, 4], forest_scores, marker="s", label="random forest")
plt.ylim(0.8, 1.02)
plt.xlabel("which split (random_state)")
plt.ylabel("test accuracy")
plt.title("Forests are higher and steadier")
plt.legend()
plt.show()
```

```
One tree: [0.889, 0.889, 0.956, 0.933, 0.956]
Forest:   [1.0, 1.0, 1.0, 0.978, 1.0]
```



The forest is better on every split, and its scores jump around less. The price: you can't read 100 trees the way you read one. You give up some explainability for accuracy.

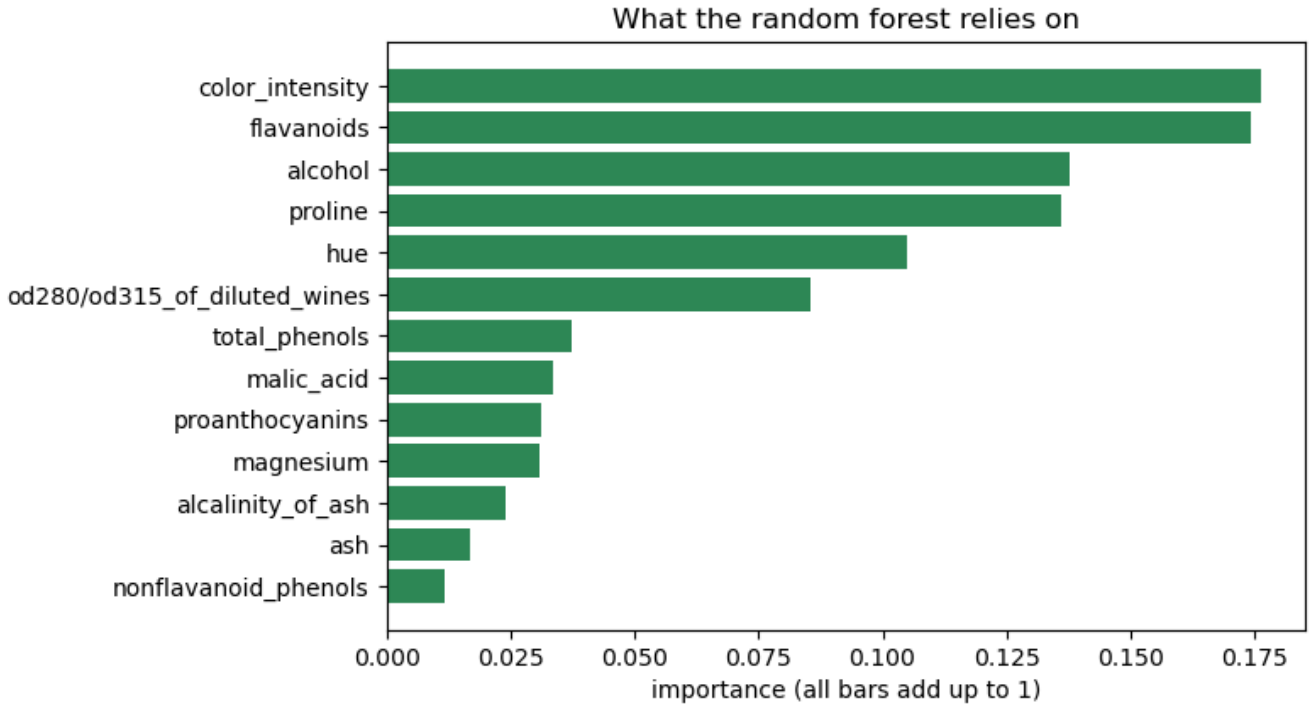
A forest still tells you which columns mattered, averaged over all its trees. This list is usually more trustworthy than the one from a single tree:

```
forest_importance = pd.Series(forest.feature_importances_, index=X.columns)
forest_importance = forest_importance.sort_values()

plt.figure(figsize=(7, 4.5))
plt.barh(forest_importance.index, forest_importance.values, color="seagreen")
plt.xlabel("importance (all bars add up to 1)")
plt.title("What the random forest relies on")
plt.show()

print("Most important column:", forest_importance.index[-1])
```

```
Most important column: color_intensity
```



Careful: importance says what the model *used*, not what *causes* the result (remember "correlation is not causation" from Day 14). When two columns carry similar information, the forest may split the credit between them.

⚠️ Common mistakes

- Calling `.predict()` before `.fit()` → `NotFittedError`.
- Celebrating 100% train accuracy. Always look at the **test** score, and at the gap between them.
- Letting a tree grow without limit on small data. Try a few `max_depth` values and compare test scores.
- Forgetting `feature_names=list(X.columns)` in `export_text` or `plot_tree`, and getting rules about `feature_0`, `feature_1`…
- Forgetting `random_state=42`: your forest (and its scores) change a little every run.

- Reading feature importance as "this causes that".

✅ Recap

- A decision tree learns a chain of yes/no questions; each **leaf** gives a prediction.
- `DecisionTreeClassifier(max_depth=…, random_state=42)`, then `fit`, `predict`, `score`.
- `export_text(tree, feature_names=…)` and `plot_tree(...)` show the learned rules.
- Train accuracy far above test accuracy = **overfitting**. Smaller depth helps.
- `RandomForestClassifier(n_estimators=100, random_state=42)`: many different trees vote; usually more accurate and steadier than one tree.
- `feature_importances_` shows which columns the model relied on (not causes).

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. A shallow tree

Create a `DecisionTreeClassifier` with `max_depth=2` and `random_state=42` in a variable called `tree`, and train it on the training data. Then store its accuracy on the **test** data in `test_acc`.

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# Create and train tree, then calculate test_acc
```

Hints:

1. `tree = DecisionTreeClassifier(max_depth=2, random_state=42)`
2. Train it with `tree.fit(X_train, y_train)`.
3. `test_acc = tree.score(X_test, y_test)`

2. Read the rules

The starting code trains a tree with `max_depth=3`. Turn it into text with `export_text`, **using the real column names**, store the text in `rules` and print it. Then read the printout and store the name of the column in the tree's **very first question** (the top line) as text in `first_question`.

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

# Make rules, print them, then set first_question
```

Hints:

1. `rules = export_text(tree, feature_names=list(X.columns))`, then `print(rules)`.
2. Run it once and look at the first line: `|--- something <= number`. The "something" is the column name.
3. Store it in quotes, for example `first_question = "alcohol"` (but with the name you actually see).

3. Depth scan

For each depth in `depths`, train a `DecisionTreeClassifier` (with that `max_depth` and `random_state=42`) and append its train accuracy to `train_scores` and its test accuracy to `test_scores`. Use a `for` loop.

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

depths = [1, 2, 3, 4, 5, 6]
train_scores = []
test_scores = []

# Loop over depths here

print("train:", train_scores)
print("test: ", test_scores)
```

Hints:

1. Start with `for depth in depths:` and indent the lines inside by 4 spaces.
2. Inside: `model = DecisionTreeClassifier(max_depth=depth, random_state=42)` and `model.fit(X_train, y_train)`.
3. `train_scores.append(model.score(X_train, y_train))` and the same with `X_test, y_test` for `test_scores`.

4. Grow a forest

Create a `RandomForestClassifier` with `n_estimators=100` and `random_state=42` in a variable called `forest`, train it, and store its **test** accuracy in `forest_acc`. It should reach at least 0.9.

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# Create and train forest, then calculate forest_acc
```

Hints:

1. It works exactly like a tree: create, `fit`, `score`.
2. `forest = RandomForestClassifier(n_estimators=100, random_state=42)`
3. `forest.fit(X_train, y_train)`, then `forest_acc = forest.score(X_test, y_test)`

5. Challenge: the most important column

Train a random forest (`n_estimators=100`, `random_state=42`) called `forest`. Then:

- make a pandas Series called `importances` from `forest.feature_importances_`, with the column names as its index, sorted from **biggest to smallest**
- store the name of the most important column in `top_feature`, taken from the Series (not typed by hand)

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# Train forest, build importances, find top_feature
```

Hints:

1. After training: `pd.Series(forest.feature_importances_, index=X.columns)`.
2. Sort it with `.sort_values(ascending=False)` so the biggest comes first.
3. The first label of a sorted Series is `importances.index[0]`.

➡️ Tomorrow

Evaluating models: so far we judged every model by accuracy. But a model that always says "healthy" can be 95% accurate and still miss every sick patient. Tomorrow you'll learn the confusion matrix, precision, recall and F1, how to choose a decision threshold, and cross-validation for a fairer score.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. A shallow tree

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X_train, y_train)
test_acc = tree.score(X_test, y_test)
print(f"Test accuracy: {test_acc:.3f}")
```

2. Read the rules

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)

rules = export_text(tree, feature_names=list(X.columns))
print(rules)
first_question = "color_intensity"
```

3. Depth scan

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

depths = [1, 2, 3, 4, 5, 6]
train_scores = []
test_scores = []

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)
    train_scores.append(model.score(X_train, y_train))
    test_scores.append(model.score(X_test, y_test))

print("train:", train_scores)
print("test: ", test_scores)
```

4. Grow a forest

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)
forest_acc = forest.score(X_test, y_test)
print(f"Forest test accuracy: {forest_acc:.3f}")
```

5. Challenge: the most important column

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

wine = load_wine(as_frame=True)
X = wine.data
y = wine.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)

importances = pd.Series(forest.feature_importances_, index=X.columns)
importances = importances.sort_values(ascending=False)
top_feature = importances.index[0]
print(importances.round(3))
print("Most important:", top_feature)
```